

MICROSERVICES · TEKNİK İNGİLİZCE · YDS

Ünite 05 · Designing business logic in a microservice architecture — Grammar Notes

Ünite bazlı çalışma kaynağı

Kaynak sırasını izle · İngilizce ve Türkçeyi karşılaştır · görsellerle tekrar et

İçindekiler

- [1. Concession — although / even though / despite](#)
- [2. Contrast — whereas / on the other hand / in contrast](#)
- [3. Relative clauses — which / that / whose](#)
- [4. Reduced relative clauses — described / implemented](#)
- [5. Passive voice — be + V3](#)
- [6. Conditionals — if clauses](#)
- [7. Purpose — in order to / so that](#)
- [8. Method — by + -ing](#)
- [9. Replacement — instead of / rather than](#)
- [10. Absence — without + -ing](#)
- [11. Embedded questions — whether / how / what](#)
- [12. Causative meaning — enable / allow / make](#)
- [13. Addition — as well as](#)
- [14. Cause and result — because / therefore / as a result](#)
- [15. Present perfect — have / has + V3](#)
- [16. Time clauses — when / once / until / while](#)
- [17. Obligation and possibility — must / should / might](#)
- [18. Degree — too ... to / enough to](#)
- [Mini quiz — Özgün çalışma soruları](#)
- [Cevaplar ve açıklamalar](#)
- [Kısa tekrar haritası](#)

Amaç: Kaynak PDF'nin 146–182. sayfalarındaki cümleleri yapı, anlam ilişkisi ve teknik bağlam bakımından çözümlmek. İngilizce örnekler belirtilen kaynak sayfalardan alınmıştır.

[Ana ders](#) · [Ünite sözlüğü](#) · [Grammar PDF](#)

Gösterimler: S = subject (özne); V = verb (fiil); V1 = yalın fiil; V3 = past participle (üçüncü biçim). Bir cümle birden fazla yapı içerebilir; başlık o örnekte odaklanılan yapıyı belirtir.

1. Concession — although / even though / despite

İşlev: Beklenenin tersine gerçekleşen durumu, “rağmen” ilişkisiyle verir.

Formül: although / even though + S + V; despite + noun / -ing

English — Kaynak örneği (s. 149): Although I'm a strong advocate of the object-oriented approach, there are some situations where it is overkill, such as when you are developing simple business logic.

Türkçe: Ben nesne yönelimli yaklaşımın güçlü bir savunucusu olsam da, basit bir iş mantığı geliştirdiğinizde gibi, aşırıya kaçan bazı durumlar vardır.

English — Kaynak örneği (s. 154): As a result, the Order is no longer valid, even though the application verified that the order still satisfied the order minimum after each consumer's update.

Türkçe: Sonuç olarak, Order artık geçerli değil, her tüketici tarafından güncelleştirildikten sonra siparişlerin hala sipariş minimumunu karşıladığını doğruladığına rağmen.

YDS ipucu: Despite of kullanılmaz. Çekimli cümle varsa although; isim grubu varsa despite düşünün.

2. Contrast — whereas / on the other hand / in contrast

İşlev: İki yaklaşımın veya aynı yaklaşımın farklı yönlerini karşılaştırır.

Formül: S + V, whereas S + V; On the other hand / In contrast, S + V

English — Kaynak örneği (s. 158): On the other hand, because an aggregate is the scope of transaction, you may need to define a larger aggregate in order to make a particular update atomic.

Türkçe: Öte yandan, bir aggregate işlem kapsamı olduğundan, belirli bir güncellemeyi atomik yapmak için daha büyük bir aggregate tanımlamanız gerekebilir.

English — Kaynak örneği (s. 181): In contrast, Order Service relies heavily on sagas when creating and updating orders.

Türkçe: Order Service, siparişler oluştururken ve güncelleştirirken saga'lar'ye büyük ölçüde güveniyor.

YDS ipucu: Whereas karşıtlık kurar; zaman bildiren when ile aynı değildir. On the other hand cümleler arasında geçiş ifadesidir.

3. Relative clauses — which / that / whose

İşlev: Bir isim hakkında tanımlayıcı veya ek bilgi verir. Önce hangi ismi nitelediğini bulun.

Formül: noun + that/which + clause; noun, which + clause

English — Kaynak örneği (s. 156): It requires that the root entity be the only part of an aggregate that can be referenced by classes outside of the aggregate.

Türkçe: entity kökeninin aggregate dışında sınıflar tarafından referanslanabilecek aggregate'nin tek parçası olması gerekmektedir.

English — Kaynak örneği (s. 166): It's a generic class that has two type parameters, A, the aggregate type, and E, the marker interface type for the domain events.

Türkçe: Bu, iki tip parametre olan bir genel sınıf, A, aggregate tipi ve E, domen olayları için işaretçi arayüz tipi.

YDS ipucu: Virgüllü ek bilgi cümleciğinde that kullanılmaz. Edat + which yapısında edatın anlamını çeviriye katın. Bir fiilin içeriğini veren “ensure that ...” ise isim niteleyen relative clause değildir; that öncesindeki yapıyı kontrol edin. İlk örnekte “requires that ...” içerik cümleciğidir; odak, “an aggregate that can be referenced ...” bölümündeki nitelemedir.

4. Reduced relative clauses — described / implemented

İşlev: İsimden sonraki V3, çoğunlukla edilgen ilgi cümleciğinin kısaltılmasıdır.

Formül: noun + (that/which + be) + V3 → noun + V3

English — Kaynak örneği (s. 155): Updating an entire aggregate rather than its parts solves the consistency issues, such as the example described earlier.

Türkçe: 'nin parçaları yerine bütün bir aggregate'yi güncelleme, daha önce açıklanan örnek gibi tutarlılık sorunlarını çözer.

English — Kaynak örneği (s. 167): KitchenServiceEventConsumer subscribes to events published by Restaurant Service whenever a restaurant's menu is updated.

Türkçe: KitchenServiceEventConsumer, bir restoranın menüsü güncellenirken Restaurant Service tarafından yayınlanan olaylara abone olur.

YDS ipucu: Cümlelerin asıl çekimli fiilini ayrı bulun. İsimden sonra gelen V3 her zaman yeni bir ana yüklem değildir.

5. Passive voice — be + V3

İşlev: İş yapan kişiden çok işlem gören şeyi öne çıkarır. Teknik metinlerde yaygındır.

Formül: S + be + V3; S + modal + be + V3

English — Kaynak örneği (s. 146): But the majority of the FTGO application's business logic is implemented in an object-oriented domain model that's mapped to the database using JPA.

Türkçe: Ama FTGO uygulamasının iş mantığını çoğunun JPA kullanarak veritabanına yerleştirilen nesne yönelimli bir alan modelinde uygulanır.

English — Kaynak örneği (s. 177): In each group, one method is invoked at the start of the saga, and the other methods are invoked at the end.

Türkçe: Her grupta, saga'nin başlangıcında bir yöntem ve sonunda diğer yöntemler kullanılır.

YDS ipucu: Must be deployed doğrudur; must deployed yanlıştır. By + kişi/araç ile by + -ing yöntem yapısını bağlamdan ayırın.

6. Conditionals — if clauses

İşlev: Sonucun hangi koşulda gerçekleştiğini bildirir. Gerçek olasılık ile varsayımı zaman biçiminden ayırın.

Formül: If + present, present / will / can + V1; If + past, would + V1

English — Kaynak örneği (s. 150): The problem is that if your business logic becomes complex, you can end up with code that's a nightmare to maintain.

Türkçe: Sorun şu ki eğer iş mantığınız karmaşıklarsa, korumak için kabus olan bir kodla sonuçlanabilirsiniz.

English — Kaynak örneği (s. 160): An Order aggregate might, if there are interested consumers, publish one of the events each time it undergoes a state transition.

Türkçe: Bir Order aggregate, eğer ilgilenmiş tüketiciler varsa, her devlete geçiş yapıldığı zaman bir olay yayınlayabilir.

YDS ipucu: If cümlecisindeki past bazen geçmiş zamanı değil varsayımı gösterir. Türkçedeki "olsaydı" tek başına zamanı belirlemez.

7. Purpose — in order to / so that

İşlev: Bir işlemin hangi amaçla yapıldığını açıklar.

Formül: in order to + V1; so that + S + can/will + V1

English — Kaynak örneği (s. 176): In order to create or update an order, Order Service must collaborate with other services using sagas.

Türkçe: Bir sipariş oluşturmak veya güncelleştirmek için, Order Service, saga'lar'yi kullanan diğer servislerle işbirliği yapmalıdır.

YDS ipucu: To sonrasında yalın fiil; so that sonrasında özne ve çekimli fiil gelir. So ... that derece-sonuç yapısıyla karıştırmayın.

8. Method — by + -ing

İşlev: Bir sonuca hangi yöntemle ulaşıldığını açıklar.

Formül: by + V-ing → ... yaparak

English — Kaynak örneği (s. 154): For example, in chapter 2 we created a rough domain model by analyzing the nouns used in the requirements and by domain experts.

Türkçe: Örneğin, 2. bölümde, gerekliliklerde kullanılan isimleri ve alan uzmanları tarafından analiz ederek kaba bir alan modeli oluşturduk.

English — Kaynak örneği (s. 147): I begin this chapter by describing the different ways of organizing business logic: the Transaction script pattern and the Domain model pattern.

Türkçe: İş mantığını düzenlemenin farklı yollarını açıklayarak başlıyorum: İşlem metni örneği ve Domain modeli örneği.

YDS ipucu: By implementing “uygulayarak” anlamındadır. Edat by sonrasında yalın fiil kullanılmaz.

9. Replacement — instead of / rather than

İşlev: Bir seçeneğin yerine başka bir seçeneğin kullanıldığını anlatır.

Formül: instead of + noun / -ing; rather than + parallel structure

English — Kaynak örneği (s. 181): Unlike in a traditional object model, references between classes in different aggregates are in terms of primary key value rather than object references.

Türkçe: Geleneksel bir nesne modelinden farklı olarak, farklı aggregate'ler'deki sınıflar arasındaki referanslar nesne referansları yerine ana anahtar değeri açısındadır.

English — Kaynak örneği (s. 147): Aggregates avoid any possibility of object references spanning service boundaries, because an inter-aggregate reference is a primary key value rather than an object reference.

Türkçe: aggregate'ler, servis sınırlarını kapsayan nesne referanslarının herhangi bir olasılığını önler, çünkü bir aggregate referansı nesne referansı yerine bir ana anahtar değeridir.

YDS ipucu: Instead of sonrasında doğrudan çekimli cümle gelmez. Rather than ile karşılaştırılan parçaların dilbilgisel biçimini izleyin.

10. Absence — without + -ing

İşlev: Bir eylem gerçekleşmeden diğer eylemin yapılabildiğini veya yapılamadığını gösterir.

Formül: without + V-ing; without + noun

English — Kaynak örneği (s. 150): You can write code without having to carefully consider how to organize the classes.

Türkçe: Sınıfları nasıl düzenleyeceğinizi dikkatlice düşünmeden kod yazabilirsiniz.

English — Kaynak örneği (s. 151): Finally, an object-oriented design is easier to extend because it can use well-known design patterns, such as the Strategy pattern and the Template method pattern, that define ways of extending a component without modifying the code.

Türkçe: Son olarak, nesne yönelimli bir tasarım genişletmek daha kolaydır, çünkü bir bileşenin kodunu değiştirmeden genişletme yollarını tanımlayan Strateji örneği ve Şablon yöntemi örneği gibi bilinen tasarım kalıplarını kullanabilir.

YDS ipucu: Without not ile otomatik birleşmez. “Without losing data”, “veri kaybetmeden” anlamındadır.

11. Embedded questions — whether / how / what

İşlev: Bir soruyu başka bir cümlemin nesnesi veya içeriği haline getirir.

Formül: verb + whether + S + V; verb + how to + V1; verb + what + clause

English — Kaynak örneği (s. 153): For example, let's look at how to ensure the order minimum is met when multiple consumers work together to create an order.

Türkçe: Örneğin, bir sipariş oluşturmak için birden fazla tüketicinin birlikte çalıştığına sipariş minimumunun nasıl yerine getirileceğini görelim.

English — Kaynak örneği (s. 147): The key decision you must make when developing business logic is whether to use an object-oriented approach or a procedural approach.

Türkçe: İş mantığını geliştirirken önemli bir karar, nesne yönelimli bir yaklaşım mı yoksa prosedürel bir yaklaşım mı kullanmak.

YDS ipucu: Dolaylı soruda düz cümle sırası kullanılır: how the service works. How does the service work doğrudan sorudur.

12. Causative meaning — enable / allow / make

İşlev: Bir işlemin başka bir eylemi mümkün kıldığını veya bir sonucu doğurduğunu anlatır.

Formül: enable/allow + object + to + V1; make + object + V1 / adjective

English — Kaynak örneği (s. 151): In addition, classes such as Account, BankingTransaction, and OverdraftPolicy closely mirror the real world, which makes their role in the design easier to understand.

Türkçe: Ayrıca, Account, BankingTransaction ve OverdraftPolicy gibi sınıflar gerçek dünyayı yakından yansıtır, bu da tasarımdaki rollerini daha kolay anlayabilmelerini sağlar.

English — Kaynak örneği (s. 153): After that, I describe the rules that aggregates must obey and how they make aggregates a good fit for the microservice architecture.

Türkçe: Bundan sonra, aggregate'ler'nin itaat etmesi gereken kuralları ve aggregate'ler'i mikroservis mimarisi için nasıl uygun hale getirdiklerini açıklarım.

YDS ipucu: Enable/allow + object + to + V1; make + object + V1/adjective yapılarını arayın. Make a request gibi make + noun kullanımları bu yapı değildir. Edilgende make ile to geri gelir: be made to do.

13. Addition — as well as

İşlev: Bir öğeye veya eyleme başka bir öğe ya da eylem ekler: "... yanı sıra".

Formül: A as well as B; as well as + noun / V-ing

English — Kaynak örneği (s. 168): Figure 5.11 shows these aggregates and other key parts of the service's business logic, as well as the service's adapters.

Türkçe: Resim 5.11 bu aggregate'ler ve servisin iş mantığı diğer önemli parçaları, yanı sıra servisin adaptörlerini gösterir.

English — Kaynak örneği (s. 171): These methods are invoked in response to REST API requests as well as events and command messages.

Türkçe: Bu yöntemler REST API isteklerine yanı sıra olaylara ve komut mesajlarına cevap olarak çağrılır.

YDS ipucu: As well as burada ekleme yapar; as fast as gibi eşitlik karşılaştırması değildir. İki as sözcüğü görünce yapıyı otomatik olarak karşılaştırma saymayın.

14. Cause and result — because / therefore / as a result

İşlev: Neden ile sonucu ayırmayı sağlar. Because neden cümlesini, therefore sonuç yargısını başlatır.

Formül: because + S + V; because of + noun; therefore / as a result + clause

English — Kaynak örneği (s. 146): Mary had encouraged her team to apply object-oriented design principles, because in her experience this was the best way to implement complex business logic.

Türkçe: Mary, takımını nesne yönelimli tasarım ilkelerini uygulamaya teşvik etmişti, çünkü deneyimine göre bu karmaşık iş mantığını uygulamanın en iyi yoluydu.

English — Kaynak örneği (s. 158): Because updates to each aggregate are serialized, more fine-grained aggregates will increase the number of simultaneous requests that the application can handle, improving scalability.

Türkçe: Her aggregate'nin güncellemeleri seriye edildiği için, daha ince taneleri olan aggregate'ler, uygulamanın ele alabileceği eşzamanlı taleplerin sayısını artıracak ve ölçeklenebilirliği iyileştirecektir.

YDS ipucu: Because ile because of sonrasındaki yapı farklıdır. As a result of + noun, neden belirtir.

15. Present perfect — have / has + V3

İşlev: Geçmişte başlayan veya tamamlanan durumun şimdiyle ilişkisini kurar.

Formül: S + have/has + V3; S + have/has + been + V3

English — Kaynak örneği (s. 152): There is one more building block that has been generally ignored (myself included!) except by DDD purists: aggregates.

Türkçe: DDD puristleri hariç, genel olarak görmezden gelen (kendim de dahil!) bir yapı taşı daha var: aggregate'ler.

English — Kaynak örneği (s. 179): The noteApproved() method is invoked when the consumer's credit card has been successfully authorized.

Türkçe: noteApproved() yöntemi, tüketicinin kredi kartı başarılı bir şekilde onaylandığında kullanılır.

YDS ipucu: Have/has ile V3 birlikte aranır. Been + V3 edilgen olabilir; been + -ing ise continuous yapıdır.

16. Time clauses — when / once / until / while

İşlev: Olayların zamanını, sırasını veya eşzamanlılığını belirtir; while bazen karşıtlık da kurar.

Formül: when/once/until/while + S + V; while + -ing

English — Kaynak örneği (s. 149): When using the Transaction script pattern, the scripts are usually located in service classes, which in this example is the OrderService class.

Türkçe: Transaction script modelini kullanırken, senaryolar genellikle servis sınıflarında yer almaktadır, bu örnekte OrderService sınıfıdır.

English — Kaynak örneği (s. 169): As described in chapter 2, when talking about the concept of a Bounded Context, this aggregate represents the restaurant kitchen's view of an order.

Türkçe: Bölüm 2'de açıklandığı gibi, bir bounded context (sınırlı bağlam) kavramından söz ederken, bu aggregate restoran mutfağının bir sipariş görünümünü temsil eder.

YDS ipucu: Geleceğe yönelik zaman cümleciğinde genellikle present kullanılır: when it arrives. Until, "... olana kadar" sınırını verir. While eşzamanlılık ya da karşılıklı bildirir; anlam ilişkisini kontrol edin. Until + noun bir zaman ifadesidir, tam zaman cümleciği değildir.

17. Obligation and possibility — must / should / might

İşlev: Zorunluluk, tavsiye ve olasılığı ayırır. Teknik gereksinimlerde bu ayrım önemlidir.

Formül: modal + V1; modal + be + V3

English — Kaynak örneği (s. 153): The FTGO application must ensure that any attempt to update an order doesn't violate an invariant such as the minimum order amount.

Türkçe: FTGO uygulaması, bir siparişi güncelleme girişiminin, en az sipariş miktarı gibi değişmez bir şeyi ihlal etmemesini sağlamalıdır.

English — Kaynak örneği (s. 161): The ID of the aggregate that emitted the event might also be part of the envelope rather than an explicit event property.

Türkçe: Olayı yayımlayan aggregate'nin ID'i de açık bir olay özelliği yerine zarfın bir parçası olabilir.

YDS ipucu: Must not yasak; do not have to zorunluluk yokluğu bildirir. Might ve may olasılık anlatır, kesinlik vermez.

18. Degree — too ... to / enough to

İşlev: Too aşırı derece nedeniyle engeli; enough gerekli yeterliliği anlatır.

Formül: too + adjective + to + V1; adjective + enough + to + V1

English — Kaynak örneği (s. 176): Otherwise, if a cancel() operation is rejected because, for example, it's too late to cancel the order, then the Order transitions back to the APPROVED state.

Türkçe: Aksi takdirde, bir cancel() işleminin reddedildiği için, örneğin, sipariş iptal etmek için çok geç olduğu için, Order yeniden APPROVED durumuna geçiyor.

YDS ipucu: Enough sıfattan sonra, isimden önce gelir: fast enough; enough memory.

Mini quiz — Özgün çalışma soruları

Parantez içindeki iki seçenekten uygun olanı seçin.

1. The message can ____ twice. (deliver / be delivered)
2. The service updates its state ____ applying the event. (by / although)
3. ____ the test passes, the pipeline proceeds. (When / Despite)

4. The adapter allows the application ____ another service. (call / to call)
5. The application runs without ____ the remote service. (contacting / contact)
6. The object ____ in this chapter is an example. (described / describes)

Cevaplar ve açıklamalar

1. **be delivered.** Mesaj teslim edilir: modal + be + V3. Deliver seçilirse mesajın teslim etme işini kendisinin yaptığı söylenir.
2. **by.** By + -ing yöntem bildirir: olayı uygulayarak. Although ardından özne ve çekimli fiil gerekir.
3. **When.** Ardından tam cümle gelir ve zaman ilişkisi kurulur. Despite isim veya -ing ister.
4. **to call.** Allow + object + to + V1 kullanılır.
5. **contacting.** Without bir edattır; ardından fiil gelecekse -ing biçimi kullanılır.
6. **described.** The object that is described kısaltılmıştır. Describes seçimi ikinci bir bağımsız yüklem oluşturur.

Kısa tekrar haritası

İlişki	Aranacak işaret	Okuma sorusu
Neden	because / because of	Neden gerçekleşiyor?
Sonuç	therefore / as a result	Bunun sonucu ne?
Koşul	if	Hangi koşulda?
Amaç	in order to / so that	Ne amaçla?
Yöntem	by + -ing	Nasıl yapılıyor?
Niteleme	which / that / V3	Hangi nesne anlatılıyor?
Edilgen	be + V3	İşlem gören şey ne?